BeBetter — Project Documentation

Nov 2, 2026 · @ghzaiel

Overview

BeBetter turns a user's soft-skills problem, written in a few sentences, into a personal programme of 5 to 10 practical missions generated by AI. The user confirms each mission as they complete it and follows their progress per programme and overall.

Main features:

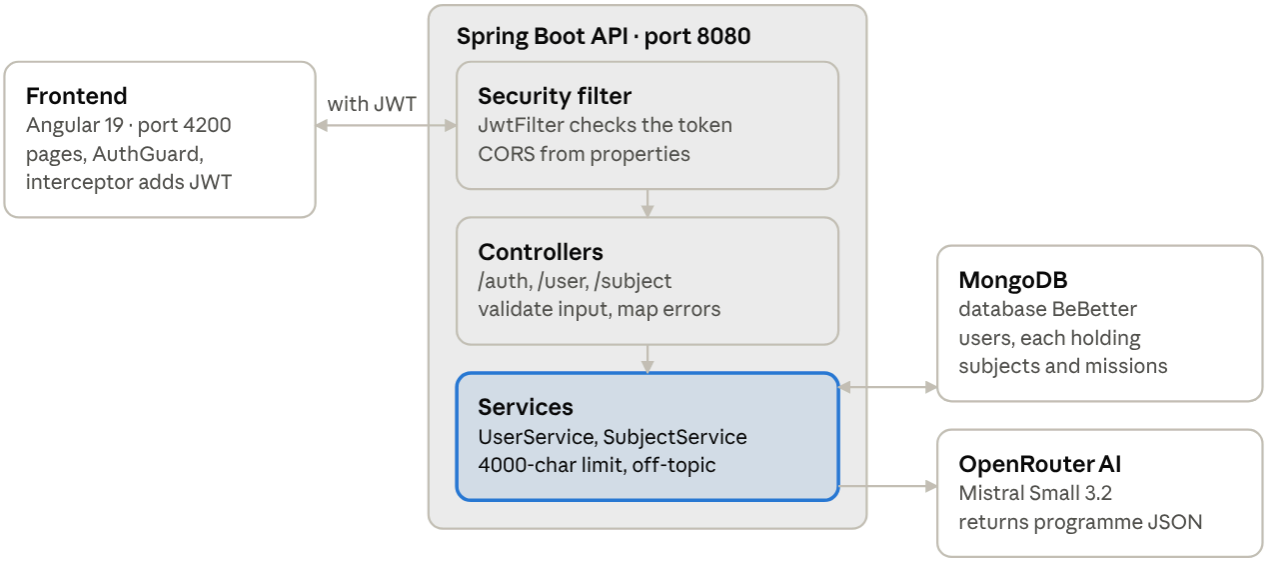
- Sign up and sign in with a JWT token.
- Generate a programme from free text (max 4000 characters) or from a suggested topic card on the home page.
- Off-topic requests (homework, code, recipes, random text) are refused with a short explanation.
- Confirm missions as done; progress bars per programme and a dashboard with overall completion.

Layer	Technology
Frontend	Angular 19, standalone components, Bootstrap 5, jwt-decode
Backend	Spring Boot 3.5.5, Java 17, Spring Security, springdoc (Swagger)
Database	MongoDB (database `BeBetter`, collection `users`)
AI	OpenRouter chat API, model `mistralai/mistral-small-3.2-24b-instruct`

Architecture

The Angular app talks only to the Spring Boot API; the API stores everything in MongoDB and calls OpenRouter to generate programmes.

The Spring API sits between Angular, MongoDB and OpenRouter



architecture · frontend, API layers, database, AI

Data is stored as one document per user: a `Utilisateur` (username, email, BCrypt password, role) holds a list of `Subject` programmes (name, description), and each subject holds its `Mission` list (name, description, `etat` = done or not). Subjects and missions have no collection of their own; they are saved with the user.

Installation and running

The backend runs on port 8080 and the frontend on port 4200; MongoDB must be running locally on 27017.

Prerequisites: Java 17, Node.js with npm, MongoDB, and an OpenRouter account with a few dollars of credit.

1. Clone the repository.
2. Create the backend configuration from the template (the real file is git-ignored because it holds the API key):
   a. Copy `bebetter-backend/src/main/resources/application.properties.example` to `application.properties` in the same folder.
   b. Create a key at openrouter.ai/keys and set `openrouter.api.key=sk-or-...`.
3. Start MongoDB.
4. Start the backend from `bebetter-backend`: `./mvnw spring-boot:run` (Windows: `mvnw.cmd spring-boot:run`), or run `BebetterBackendApplication` from IntelliJ.
5. Start the frontend from `BEBETTER_FRONT`: `npm install`, then `npm start`.
6. Open `http://localhost:4200`, sign up, and generate a first programme.

Swagger UI is available at `http://localhost:8080/swagger-ui.html` once the backend is up.

Configuration reference

All backend settings live in `application.properties`; changing any of them needs a backend restart, not a code change.

Property	Default value	Purpose
`spring.data.mongodb.uri`	`mongodb://localhost:27017/BeBetter`	MongoDB connection and database name
`logging.level.org.springframework.security`	`DEBUG`	Security log detail; set to `INFO` for a quieter console
`openrouter.api.url`	`https://openrouter.ai/api/v1/chat/completions`	OpenRouter chat endpoint
`openrouter.api.key`	none (secret)	Your OpenRouter key, sent as a Bearer header
`openrouter.model`	`mistralai/mistral-small-3.2-24b-instruct`	AI model used to generate programmes
`app.prompt.subject-coach`	`classpath:prompts/subject-coach-prompt.txt`	File holding the coach's system prompt
`app.cors.allowed-origins`	`http://localhost:4200`	Front-end origins allowed by CORS, comma-separated

The frontend calls the backend at `http://localhost:8080`, written in `auth.service.ts` and `programme-service.service.ts`. The 4000-character prompt limit is set in two places that must match: `SubjectService.MAX_PROMPT_LENGTH` and `maxPromptLength` in `home.component.ts`.

Security

Every request except sign-in, sign-up and Swagger needs a valid JWT in the `Authorization: Bearer <token>` header.

- **Tokens:** `/auth/login` and `/auth/register` return a JWT signed with HS512, valid for 10 hours, with the username as subject. The frontend stores it in `localStorage` and the `AuthInterceptor` adds it to every request.
- **Roles:** new accounts always get `ROLE_USER`; the server ignores any role sent by the client. An admin is made by setting `role` to `ROLE_ADMIN` directly in MongoDB. The `ROLE_` prefix is required by Spring's `hasRole()` checks.

- **CORS:** configured only in `SecurityConfig`, from the `app.cors.allowed-origins` property. Controllers no longer carry `@CrossOrigin`.
- **Passwords:** hashed with BCrypt; at least 6 characters at sign-up.

Path	Method	Access
`/auth/login`, `/auth/register`	POST	Public
`/swagger-ui/**`, `/v3/api-docs/**`	GET	Public
`/subject/**`	All	`ROLE_USER` or `ROLE_ADMIN`
`/user/all`	GET	`ROLE_ADMIN`
`/user/delete/**`	DELETE	`ROLE_ADMIN`
`/user/update/**`	PUT	`ROLE_ADMIN`
Anything else	All	Any signed-in user

REST API reference

Base URL: `http://localhost:8080`. Errors from the auth and programme-creation endpoints come back as `{ "message": "..." }`.

Auth

Method	Path	Body	Success	Errors
POST	`/auth/register`	`{ username, email, password }`	201 `{ token }`	400 missing field or password under 6 characters; 409 username taken
POST	`/auth/login`	`{ username, password }`	200 `{ token }`	401 invalid credentials

Programmes (subjects and missions)

Method	Path	Purpose	Success
POST	`/subject/newSubjects/{username}`	Generate a programme; body = the raw prompt text	200, the updated user
GET	`/subject/getAllSubjects/{username}`	List the user's programmes	200, list of subjects
GET	`/subject/getSubject/{username}/{subjectId}`	One programme with its missions	200, or 404
GET	`/subject/getMissions/{username}/{subjectId}`	Missions of one programme	200, list of missions
PUT	`/subject/done/{username}/{subjectId}/{missionId}`	Mark a mission as done	200
DELETE	`/subject/deleteMission/{username}/{subjectId}/{missionId}`	Delete one mission	200
DELETE	`/subject/deleteSubject/{username}/{subjectId}`	Delete a programme	200

Programme creation errors: 400 when the prompt is empty or over 4000 characters, 422 `{ message, offTopic: true }` when the AI judges it off-topic, 500 when the AI call fails.

Users

Method	Path	Purpose	Access
GET	`/user/{username}`	User by username	Signed-in
GET	`/user/byId/{userId}`	User by id	Signed-in
GET	`/user/all`	All users (204 when empty)	Admin
PUT	`/user/update/{userId}`	Update username, email or password	Admin

Method	Path	Purpose	Access
DELETE	`/user/delete/{userId}`	Delete a user	Admin

AI programme generation

One programme costs about $0.0003 with the default model, so $1 of OpenRouter credit covers roughly 3000 programmes.

1. The backend checks the prompt: not empty, at most 4000 characters (otherwise 400, and no AI call is made).
2. It sends two messages to OpenRouter: the coach instructions from `prompts/subject-coach-prompt.txt` as the system message, and the user's text as a separate user message. Keeping them apart makes it harder for a user to override the rules.
3. The request asks for JSON only (`response_format: json_object`).
4. The model answers either a programme `{ name, description, missions[] }` or `{ offTopic: true, reason }` .
5. Off-topic → 422 with the reason, nothing is saved. Otherwise the subject and its missions (all `etat = false`) are added to the user in MongoDB.

Editing the prompt: change `src/main/resources/prompts/subject-coach-prompt.txt` and restart the backend. The JSON shape it asks for must stay the same, since the code reads `name` , `description` , `missions[].name` , `missions[].description` , `offTopic` and `reason` .

Changing the model: set `openrouter.model` . Prices fetched from OpenRouter on 2 Nov 2026:

Model	Input ($ / M tokens)	Output ($ / M tokens)	Note
`mistralai/mistral-small-3.2-24b-instruct`	0.075	0.20	Default: good French/English and JSON
`mistralai/mistral-nemo`	0.029	0.03	Cheapest, weaker answers
`openai/gpt-4o-mini`	0.15	0.60	About twice the default's price
`google/gemini-2.5-flash-lite`	0.10	0.40	Close alternative

Off-topic detection is decided by the AI, so borderline prompts can occasionally be accepted or refused wrongly.

Frontend

All pages except sign-in and sign-up are protected by `authGuard`, which redirects to `/auth/login` when no token is stored.

Route	Component	What it does
`/auth/login`	`LoginComponent`	Sign in; links to sign-up
`/auth/register`	`RegisterComponent`	Sign up with username, email, password and confirmation; signs the user in and opens `/home`

Route	Component	What it does
`/home`	`HomeComponent`	Prompt box with a 4000-character counter, error messages (too long, off-topic) and suggested topic cards
`/about-us`	`AboutUsComponent`	About page
`/programmes/layout/allprogrammes`	`AllProgrammesComponent`	All programmes, each with a progress bar, expandable missions and an "Open programme" link
`/programmes/layout/programme/:id`	`ProgrammeDetailComponent`	One programme: progress bar, numbered missions and a "Confirm done" button on each
`/programmes/layout/dashboard`	`ProgrammeDashboardComponent`	Overall completion %, missions done, programmes finished, and a progress bar per programme

Key services and helpers:

- `AuthService`: `login()`, `register()`, `logout()`, `getUsername()` (read from the token, so it works right after a page refresh), current user state.
- `ProgrammeServiceService`: one method per `/subject` endpoint.
- `progress.util.ts`: `subjectProgress()` and `overallProgress()` return done, total and percent, used by the list, detail page and dashboard.

Confirming a mission asks for confirmation first, because the backend has no way to mark a mission as not done again.

Troubleshooting and known limitations

Symptom	Cause	Fix
"Web server failed to start. Port 8080 was already in use"	Another app (e.g. another Spring project) is using 8080	Stop that app, or set `server.port` and update the frontend URLs
Console full of DEBUG lines	`logging.level.org.springframework.security=DEBUG`	Set it to `INFO`
403 on `/subject/**` for an old account	Old accounts store `USER_ROLE` instead of `ROLE_USER`	Set `role` to `ROLE_USER` in MongoDB, or create a new account
Programme creation returns 500	Missing or invalid OpenRouter key, or no credit	Check `openrouter.api.key` and your OpenRouter balance; the backend log prints the OpenRouter error
Backend does not start, prompt file error	`app.prompt.subject-coach` points to a missing file	Restore `prompts/subject-coach-prompt.txt`
CORS error in the browser	Front-end origin not allowed	Add it to `app.cors.allowed-origins`

Known limitations:

- A mission cannot be marked as not done again; there is no undo endpoint.
- The sidebar's "Create New Program" link has no page yet.
- The `/subject` and `/user/{username}` endpoints trust the username in the URL, so a signed-in user can read or change another user's programmes. They should use the username from the token instead.
- The JWT signing secret is written in `JwtUtil.java`; it should move to configuration.
- The old Gemini API key is still readable in the git history and should be revoked.